

JSON in Artificial Intelligence: Specification and Applications

Introduction

JavaScript Object Notation (JSON) is a **lightweight, text-based, language-independent data interchange format** derived from JavaScript ¹. It defines a minimal set of syntax rules for representing structured data in a portable way ¹. JSON has become ubiquitous across computing, and it plays a significant role in artificial intelligence (AI) workflows. From dataset annotation files to model configuration and RESTful API responses, JSON is commonly used to store and transmit data in AI systems. This paper provides an academic overview of JSON's formal specifications and examines its usage in AI. We discuss JSON's syntax and structure, its applications in data storage, training pipelines and model deployment, a comparison with PDF and plain text formats, and considerations of performance, scalability, and security in AI contexts. Relevant literature and official specifications (e.g., IETF RFC 8259) are cited to ground the discussion in authoritative sources.

JSON Syntax and Structural Rules

JSON Data Types: JSON represents *six* data types: four primitives and two structured types. According to the JSON standard, the primitive types are **strings**, **numbers**, **booleans**, and **null**, and the structured types are **objects** and **arrays** ². An *object* in JSON is an unordered collection of name/value pairs, delimited by curly braces `{ }`. Each name is a string and is separated from its value by a colon, and pairs are separated by commas ³. An *array* is an ordered sequence of values, enclosed in square brackets `[ ]` and separated by commas ³. JSON strings are sequences of Unicode characters, typically wrapped in double quotes, with support for escape sequences (for special characters or encoding Unicode by code point) ² ⁴. Numerical values in JSON are expressed in decimal notation (with optional fractional and exponential parts) and do not include machine-specific notation like `NaN` or `Infinity` (these are not valid JSON literals) ⁵. Boolean values are represented as the literals `true` or `false`, and the literal `null` represents an empty value ².

Syntax Rules and Encoding: JSON syntax is defined by a small set of structural characters: braces `{ }`, brackets `[ ]`, commas `,`, and colon `:` ⁶ ⁷. These characters, along with whitespace, are used to delimit the data structures. Insignificant whitespace (spaces, tabs, newlines) is allowed between tokens, enabling pretty formatting without altering meaning ⁸. **Trailing commas are not permitted** after the last item in an object or array; JSON's grammar treats any trailing comma as a syntax error ⁹. This strictness means that each comma in JSON must be followed by another value, enforcing a clear end to lists. JSON's design favors unambiguous parsing at the cost of some flexibility (for example, comments are not allowed in pure JSON, unlike some configuration file formats). All JSON keys (object member names) must be quoted strings. For instance, a valid JSON object could be:

```
{
  "name": "Alice",
  "age": 30,
  "is_student": false,
  "courses": ["Math", "Computer Science"]
}
```

In terms of encoding, JSON texts exchanged between systems **must be encoded in UTF-8** for maximum interoperability ¹⁰. The IETF JSON specification (RFC 8259) mandates UTF-8 encoding for JSON in open systems, reflecting the reality that nearly all JSON implementations have converged on UTF-8 ¹⁰. This ensures consistent interpretation of Unicode characters across platforms. JSON is typically transmitted with the MIME media type `application/json` ¹¹.

Formal Specification: JSON's formal grammar is defined in RFC 8259 and ECMA-404. The JSON text is essentially a serialized value (allowing the top-level value to be an object, array, string, number, etc.). Earlier JSON specifications required the top-level to be an object or array, but modern JSON allows any value at the top for flexibility ¹². The grammar uses a precise lexical definition for numbers (e.g. an optional minus sign, followed by digits, optionally a decimal point and fraction, and an exponent part) ¹³. This means that JSON numbers are typically treated as IEEE 754 double precision floats in implementations, which has implications for precision and range ¹⁴. Indeed, **JSON cannot natively represent special numeric values** like Not-a-Number or infinity, since those tokens are not defined in the syntax ⁵. Any attempt to include such values (e.g., from a computation that produces `NaN`) requires workarounds like quoting the value as a string or using `null`, which complicates encoding and decoding logic. These limitations underscore that JSON's simplicity comes with certain expressive trade-offs.

JSON in Data Storage and Communication

JSON's simplicity and portability have made it a popular choice for data storage formats and communication protocols. Many systems use JSON for storing configurations, logging information, or even as a lightweight database format. For example, **document-oriented databases** like *MongoDB* use JSON-like documents as the primary data model. In practice, MongoDB stores data internally in a binary JSON derivative (BSON) for efficiency, but exposes a JSON interface to developers ¹⁵. This allows data to be stored and queried as rich objects (with nested fields, lists, etc.) in a way that is both human-readable and machine-parseable. JSON files are also commonly used to persist structured data on disk – for instance, configuration files (`config.json`), application settings, or serialized objects. Unlike binary serialized formats, JSON text files can be opened in a text editor and understood at a glance, aiding in debugging and manual editing when necessary.

In data science and AI, it is common to use JSON for **dataset storage and annotations**. A prominent example is the *COCO dataset* (a standard benchmark for computer vision), which uses JSON files to store image annotations (bounding boxes, labels, etc.) for thousands of images. The COCO format has become a de facto standard for object detection and image segmentation tasks. As one source notes, “*COCO is a common JSON format used for machine learning*”, having gained popularity through the widespread use of the COCO dataset as a benchmark ¹⁶. In the COCO JSON schema, top-level fields include lists of `images`, `annotations`, and `categories`, each containing structured records; this illustrates JSON's capability to represent complex, relational data (multiple linked lists of objects) in a single file. Many other AI datasets

adopt similar JSON-based formats or provide JSON exports, because JSON allows encoding rich annotation information (labels, coordinates, metadata) in a single self-describing document.

JSON is also a cornerstone of modern **web communication protocols**. In particular, JSON has essentially replaced XML as the preferred format for data in RESTful APIs and web services. A 2023 industry comparison observed that *“most public web services provide REST APIs and transfer data in the compact and easy-to-use JSON data interchange format”* ¹⁷. When an AI model is served via a web API (for example, a sentiment analysis model behind a REST endpoint), the input is often submitted as JSON (encapsulating features like text or image data, possibly encoded as Base64 if binary) and the output predictions are returned as JSON. The popularity of JSON in APIs comes from its conciseness and native support in JavaScript, which allows direct consumption by web front-ends. Even in microservice architectures within AI systems, internal services often communicate via JSON over HTTP because it's language-agnostic and human-readable for troubleshooting.

Configuration and Logging: In AI training pipelines, JSON is frequently used to configure experiments or log results. Many machine learning platforms and libraries allow saving parameters, metrics, or model metadata as JSON. For example, an experiment tracking system might store each run's settings (learning rate, model type, dataset name) in a JSON record. JSON Lines (NDJSON) format, where each line is a separate JSON object, is often used for logging streaming data or large lists of records incrementally. JSON's text nature means logs can be grepped and parsed with standard text processing tools, while still being easily loadable into Python/R data structures for analysis.

JSON in AI Model Training Pipelines

Within model training workflows, JSON finds use in multiple capacities: as a format for **data annotation**, for **dataset interchange**, and for **model or experiment configuration**. One key advantage of JSON in these contexts is that it can represent hierarchical or nested information naturally, which is common in AI data.

Data Annotations: Many supervised learning tasks require annotated data (labels, bounding boxes, etc.). JSON is often used to store such annotations alongside the raw data. For instance, in image recognition or object detection, a single JSON file can contain an array of objects each describing one image's metadata and labeled regions. Because JSON supports nesting, a complex annotation (with an image's filename, size, and a list of multiple labeled objects each with coordinates and categories) can be expressed in a clear tree structure. The **COCO format** mentioned earlier is a paradigmatic example: it uses nested objects for images and their annotations. This structured approach is more robust and extensible than a flat CSV or plain text – new fields can be added to the JSON schema without breaking compatibility, as non-required keys can be ignored by older tools. Other domains also use JSON for annotations: for example, in natural language processing, one might use JSON lines where each line contains a JSON object with a text sample and its annotations (such as entity tags or sentiment), allowing easy reading and writing by scripts.

Configuration Files: JSON is employed for configuring machine learning experiments and pipelines. Hyperparameter settings, model architecture choices, or data preprocessing options can be written in a JSON config file that the training script reads. JSON's syntax (while lacking comments and more advanced features) is simple and forces a clear structure of key-value pairs, which encourages well-defined configuration schemas. In scenarios where users need to modify parameters, JSON's human-readability is a benefit. Some frameworks support JSON or a JSON-derived syntax for configuration; for example, TensorFlow's `tf.train.Checkpoint` or some PyTorch libraries can export settings to JSON. In

reinforcement learning or complex pipelines, one might have a `config.json` that enumerates environment settings, network layers, etc., which the code parses at startup. While YAML has grown popular for configurations due to its readability and ability to include comments, JSON remains widely used especially where strict syntax and interoperability are priorities.

Model Architecture and Metadata: Certain AI frameworks offer JSON serialization for model structures. Notably, **Keras (TensorFlow)** provides a `model.to_json()` method that saves a neural network's architecture (layer types, sizes, connections) in JSON format¹⁸. This JSON can later be read to reconstruct the model (`model_from_json()`), facilitating model sharing without weights. As Brownlee (2023) describes, *"Keras provides the ability to describe any model using JSON... This can be saved to a file and later loaded... to create a new model from the JSON specification."*¹⁸. This usage highlights JSON's role in AI for *serialization*: it can capture complex in-memory objects (like neural network graphs) in a standard text form. Similarly, JSON often appears in metadata files accompanying trained models—e.g., a `model_card.json` containing information about a model's intended use, or a `vocab.json` mapping tokens to indices in NLP models. These JSON files are easy for other programs or services to consume when integrating the model into a larger system.

Dataset Exchange: When sharing datasets, JSON is a common denominator format because it can be parsed in virtually any programming language. Annotations in JSON (as opposed to a binary format) enable researchers to quickly inspect and modify the data. Many open datasets in AI are distributed with JSON metadata or use JSON-based standards (for instance, many NLP corpora on platforms like Hugging Face Datasets include a JSON or JSONL version). The self-describing nature (each value is paired with a field name) reduces ambiguity, which is valuable when different teams or tools use the data.

JSON in Model Deployment and Inference

In deploying AI models for inference (whether in a cloud service or embedded application), JSON is frequently used for **input and output formats**. This is especially true when models are deployed behind RESTful APIs. For example, *TensorFlow Serving*, a popular system for serving trained models, provides a REST API where **requests and responses are JSON objects**¹⁹. The request JSON might contain inputs (e.g., feature values or an encoded image) under predefined keys, and the response JSON contains the model's predictions or outputs in a structured form. The TensorFlow Serving documentation specifies that the REST response will be a JSON representation of the underlying gRPC response message, and even error messages are returned as JSON with an `"error"` field¹⁹. This design allows easy integration with web clients and simplifies debugging (since the output can be read as text).

Consider an example of a deployed image classification model: a client might send an HTTP POST with a JSON body like `{"image_data": "<base64-encoded image>"}`, and receive a JSON reply such as `{"predictions": [{"label": "cat", "confidence": 0.98}, {"label": "dog", "confidence": 0.01}]}`. This response is straightforward for a calling application to parse and use. Many cloud AI services (Google Cloud AI, AWS SageMaker, etc.) also default to JSON for REST interfacing, because it cleanly encapsulates complex data (like a list of predictions, each with multiple attributes) and is supported by standard HTTP libraries.

Web and Mobile Integration: When AI models are accessed from web or mobile apps, JSON is the natural choice for data exchange. JavaScript in web pages can directly consume JSON via `fetch` or AJAX calls and

manipulate the data as native objects, given that JSON is a subset of JavaScript. On mobile devices and IoT devices, JSON's text format is lightweight enough for network transmission and can be parsed by built-in libraries (e.g., many languages have a simple JSON decoder). Notably, an analysis of JSON traffic on a content delivery network found that *“mobile applications account for at least 52% of JSON traffic ... and embedded devices another 12%”*, meaning nearly 2/3 of JSON data is parsed on constrained devices rather than full web browsers ²⁰. This underlines why JSON's ease of use across platforms is critical for AI deployment: a model's output in JSON can be readily used by a smartphone app or an edge device without custom parsing logic.

Interoperability and Standardization: JSON as an interface format also aids interoperability between different components of an AI system. For instance, a model might be written in Python and served with a small Flask API that returns JSON; a Java application or a JavaScript front-end can consume this output easily. Because JSON is standardized (RFC 8259 / ECMA-404) and mature, there is little ambiguity in how data is encoded. The use of JSON schemas or API documentation (like OpenAPI/Swagger for REST endpoints) can further enforce the expected structure of JSON inputs/outputs for AI services, enabling automatic client code generation and validation. In inference workflows, JSON might also be used for **pipeline messaging** – if an AI system has multiple stages (say, a text extraction stage followed by an ML classifier), they might pass data as JSON messages through a queue or streaming platform.

Comparison to Alternative Formats: For model deployment, an alternative to JSON sometimes used is Protocol Buffers (Protobuf) or other binary formats (especially with gRPC). While binary protocols can be more efficient, JSON's human-readability and web-friendliness often outweigh performance concerns for many applications. JSON enables developers to manually inspect API calls and logs, and to use a wide array of tools (like `curl` or Postman) to interact with the model endpoints. In the context of explainable AI, JSON is also useful for returning rich, nested explanations (for example, a model might return not just a prediction but also a hierarchy of contributing factors or a tree of feature importances in JSON form). These can be arbitrarily complex structures, but as long as they remain within JSON's expressive capabilities (objects, arrays, and primitive values), they can be transmitted and visualized or processed by downstream consumers.

Technical Comparison: JSON vs PDF vs Plain Text

When considering data formats in AI workflows, JSON occupies a unique position as a structured text format, distinct from binary document formats like PDF or unstructured plain text files. Here we compare JSON with **PDF** and **TXT (plain text)** in terms of structure, readability, parsing, and suitability for AI applications:

- **Structure:** JSON is **highly structured** – it explicitly encodes hierarchy (through nesting of objects/arrays) and key-value relationships. Each data element is annotated with a key (in objects) or position (in arrays), yielding self-describing data. In contrast, **PDF (Portable Document Format)** is *primarily a document layout format*, aimed at faithfully rendering text and graphics. Internally, a PDF file does have a structure (sections for objects, streams, cross-reference tables, etc.), but this structure is geared toward page description, not toward semantic data representation. The content in a PDF is essentially fixed on pages, often without machine-readable demarcation of logical units like paragraphs or tables (unless tagged PDFs are used). As Montani (2024) quips, *saying “I have the data in a PDF” is about as meaningful as saying “I have it on my computer” – it can mean anything* ²¹. This highlights that PDF encapsulates information in a visually formatted way that **lacks a predefined**

schema for the actual data – text could be anywhere, in any order, possibly embedded in images or complex layouts. **Plain text (TXT)** files, on the other hand, contain raw sequences of characters with minimal structure (maybe newlines or simple delimiters). A `.txt` file might hold unstructured data (like a transcript or log) or lightweight structured data (CSV or TSV are essentially plain text with delimiters). But plain text has **no inherent hierarchy** or keys: any structure is imposed by conventions or external agreement (e.g., a line break separating records, or a specific column order). In summary, JSON provides a **rich, nested data model**, whereas PDF is a **formatted document container** and TXT is **raw text with no imposed structure**.

- **Readability:** JSON is designed to be **human-readable** (one of its design goals is being textual and relatively clear) ²². A JSON snippet can be understood by a developer: keys provide semantic context, and the format uses common punctuation. However, for very large JSON files, readability can decline without pretty-printing or grouping. **Plain text** is human-readable if the content is natural language (e.g., a paragraph of English) or simple data, but if used to encode data (say, a list of values separated by spaces), it may be harder to interpret without knowing the format. Plain text is easily viewable and editable in any text editor, and it's compact for simple data, but it doesn't self-describe the meaning of each part. **PDF files** are *not intended to be read as raw data*; they are meant to be read visually via a PDF viewer. The underlying PDF content is binary/ASCII mixed and typically compressed, so opening a PDF in a text editor yields gibberish aside from a few header lines. In effect, PDFs are human-readable only *after* rendering – they excel at presenting readable content to humans (fonts, images, layout), but not at exposing that content in a way a person or simple script can parse. In AI workflows, this means that if one is given information in PDF form (say a dataset of scientific articles in PDF), an extra OCR or parsing step is needed to make that information machine-usable, whereas JSON or structured text could be read directly. The readability advantage of JSON (and plain text) also aids **transparency**: one can open a JSON config or output and inspect it, which is not possible with a PDF without specialized tools.
- **Parsing and Data Extraction:** JSON is straightforward to parse by machines – virtually all programming languages offer JSON parsers that convert JSON text into native data structures (e.g., dictionaries, lists, etc.). The parsing process is unambiguous due to JSON's strict grammar. JSON's structure means that once parsed, data can be accessed via keys or indices, making extraction of specific fields trivial. **Plain text** requires custom parsing logic or use of simple delimiters. For example, if data is stored as comma-separated values in a `.txt` file, a developer must split lines by commas and know which column is which (or use a CSV library). If the plain text is unstructured (like free-form text), extracting specific information (e.g., numbers or names) might require regular expressions or AI methods (like NLP techniques). **PDF parsing** is notably complex: as one technical blog on NLP and PDFs notes, *“processing PDFs is hard”* ²³. A PDF parser must interpret the page structure, text blocks, fonts, and sometimes even perform OCR if the text is embedded as an image. Extracting a table or a figure caption from PDF programmatically can be challenging because the format doesn't guarantee a logical reading order or explicit tags (unless the PDF is tagged, which many are not). For AI workflows that need to get data out of PDFs (e.g., text for language modeling or forms for information extraction), considerable effort is spent on parsing – using libraries like PDFBox, PyPDF, or advanced layout understanding models. In contrast, extracting data from JSON is trivial in comparison. As a result, **JSON is far more suitable as an input/output format for AI pipelines**, whereas PDFs often represent raw data sources that need conversion to text/JSON for analysis. Montani (2024) emphasizes *getting your data out of PDFs as early as possible* in an ML pipeline, converting it to a structured form for exactly this reason ²¹.

- **Suitability for AI Workflows:** JSON is **well-suited for AI** in roles where structured data exchange is needed. During model training, JSON can provide input annotations or configuration in a readily digestible way. During inference, JSON serves as a lingua franca between services and as a logging format. AI systems also produce outputs (predictions, explanations) that are naturally structured – JSON captures these without losing structure. For example, a complex prediction output (say, a hierarchy of categories detected in an image) can be returned as a nested JSON object. **Plain text** is most suitable in AI primarily for scenarios like training NLP models (where raw text is the substance to learn from). Large language models are often trained on massive plain text corpora (e.g., web text) because the goal is to learn language patterns from unstructured text. In such cases, plain text is indeed the data itself, and JSON would be an unnecessary wrapper. However, even in NLP, if one has labeled data (like sentences with labels or QA pairs), these are often stored in JSON or CSV rather than pure plain text, to include the labels and IDs. Plain text also appears in simpler logging or quick outputs, but it lacks standardization – each plain text dataset might have its own format that needs to be interpreted. **PDFs** have niche roles in AI: they are often sources of data (many documents are only available as PDF scans) or used for reporting results (an AI system might generate a PDF report for human end-users). They are not used *internally* for computation due to the parsing difficulty discussed. In terms of *performance*, JSON and plain text are lightweight text that can be streamed and processed incrementally, whereas PDFs are heavier (often containing fonts, images, and requiring rendering logic). In terms of *storage and versioning*, JSON is amenable to line-by-line diffing and version control (one can track changes in a JSON config easily), but PDF is binary and changes cannot be diffed in a meaningful way. **Security-wise**, JSON and plain text do not inherently carry active content (though JSON can contain scripts as data, it won't execute), whereas PDFs can embed scripts or macros (PDFs support JavaScript for interactive forms, which can pose security risks if mishandled).

In summary, **JSON** provides structured, machine-friendly organization with moderate human readability, making it ideal for the *intermediate and exchange* format in AI workflows. **Plain text** is unstructured but human-centric, suitable for raw language data or very simple outputs, but it requires additional conventions for structured information. **PDF** is oriented toward human consumption and publication-quality output, not for computational ingestion – it's an *endpoint format* rather than a working format for AI data processing²¹. AI practitioners typically convert PDFs to text/JSON, whereas they often convert JSON data into more human-friendly formats only at the final presentation stage if needed.

Performance and Scalability Considerations

Using JSON in AI systems offers flexibility and readability, but it also raises performance and scalability considerations, especially as data sizes grow. JSON is a textual format, which means it is **less compact and slower to parse** than some binary alternatives. Every character (including structural symbols and repeating key names) contributes to the payload size. For small messages (e.g., a single inference result), this overhead is negligible. However, for large datasets or extremely high-throughput systems, JSON's verbosity can become a bottleneck.

Parsing Overhead: JSON parsing requires reading and interpreting every byte in the text. McCarthy (2024) points out that “*you can't parse [JSON] without looking at every individual byte... your best parse speed is limited by your CPU's data bus throughput plus the time to classify each byte*”²⁴. Unlike binary formats (e.g., Protocol Buffers, Avro, or MessagePack) that include length prefixes or fixed offsets allowing parsers to jump through data, JSON has to be processed sequentially. This can make JSON parsing CPU-intensive when

dealing with very large JSON structures. In AI contexts, consider a scenario of reading a JSON file with millions of entries (perhaps a large image annotations file or a JSONlines corpus): the JSON parser will use significant CPU and memory to construct in-memory objects. High-performance JSON libraries (like Google's `simdjson` or `ultrajson`) have been developed to mitigate this, using SIMD instructions to speed up byte scanning ²⁵, but they still operate within the fundamental cost of processing text. The **space inefficiency** is also notable for numeric data: a number like 3.141592 might be stored as the ASCII characters "3.141592" (7 bytes) in JSON, whereas in binary IEEE754 it would be 8 bytes but convey a much larger range of values; more extremely, a large integer could take dozens of bytes in JSON (as digits) but only 4 or 8 bytes in binary form. This matters when AI systems handle large arrays or tensors – JSON is not ideal for large numeric matrices due to size and parse time. Researchers often switch to binary formats (like NumPy's `.npy` or HDF5) for heavy numeric data, while using JSON for metadata or small configuration pieces.

Memory Usage and Streaming: A naive approach to JSON is to load the entire document into memory and parse it into objects. This can be problematic if the JSON file is huge (e.g., a 10GB JSON file of training data). One advantage of JSON being text is that it can be *streamed*: tools and libraries can parse JSON incrementally (SAX-style parsing or iterative parsing) without holding it all in memory. Moreover, the community has introduced conventions like **NDJSON (Newline-Delimited JSON)** for handling large data streams. In NDJSON, each line is a separate JSON object, so a parser can read line by line (or record by record) without ever needing the whole file at once ²⁶. This is especially useful in big data and AI data pipelines, where you might process records sequentially (map-reduce style) from a source. NDJSON also plays well with line-oriented processing tools (you can, for instance, use Unix `grep` or `head` on it). Many AI data pipelines (e.g., streaming logs of predictions, or incremental training data ingestion) use JSON Lines for scalability. If a standard JSON array of millions of items were used instead, the entire array must be parsed as one structure, which is less streaming-friendly. Thus, for **scalability**, adopting streaming patterns or chunking for JSON is important. Some web APIs for AI also use streaming JSON (sending one JSON per line over a socket or HTTP response) to continuously feed predictions.

Throughput and Latency: For real-time AI services (like a model serving 1000 requests per second), JSON serialization/deserialization becomes part of the latency. Typically, JSON encoding/decoding is fast enough (< microseconds to milliseconds) for moderate payloads (a few KB), but it can become a limiting factor at extreme scale. In such cases, alternatives might be considered: e.g., using binary `ProtoBuf` over `gRPC` for inter-service communication to reduce latency, or using simpler encoding for arrays (some APIs allow CSV for large arrays of predictions since CSV is more compact for pure numeric data). That said, the convenience and ubiquity of JSON often win out until performance profiling proves it a bottleneck. Modern JSON libraries in C++, Java, or Python (with C extensions) can parse tens of MBs per second per core, which is usually adequate.

One interesting observation in a CDN study was that as JSON payloads got smaller on average (due to more granular API calls), the relative overhead of parsing became higher per byte ²⁷. Many AI systems now favor sending smaller JSON messages more frequently (microservices, micro-batching of data) rather than one giant JSON, which can actually *increase* total parsing cost system-wide. This suggests a potential trade-off: sending larger JSON blobs might be more efficient in terms of parse/setup overhead amortization, whereas many tiny JSON messages incur repeated overhead (each message has some fixed processing cost). System designers need to consider how they batch data in JSON – e.g., an inference service might accept an array of 100 inputs in one JSON request to improve throughput, rather than 100 separate JSON requests, if latency allows.

Comparison with PDF and TXT performance: Plain text (TXT) is simpler to parse than JSON if the format is trivial (e.g., one data point per line with a single value). There's almost no structural overhead, so it can be read very quickly. However, the moment the text contains complex structured data, parsing it may involve more custom logic that can be error-prone and slow (e.g., splitting by various delimiters, dealing with escaping if needed). JSON's advantage is that a single-pass parse gives you the data structure ready to use, whereas naive text parsing might iterate multiple times or use regex (which can be slower). PDF, being a binary format for documents, typically requires *much* more processing than JSON to extract data (text extraction from PDF can involve decoding font encodings, interpreting vector graphics for layout, etc.), and is orders of magnitude slower than parsing equivalent text from JSON. For example, extracting all text from a PDF document (say for NLP) might take seconds and considerable CPU, whereas reading the same text from a JSON or TXT takes milliseconds. Thus, in performance-critical AI tasks, PDF input is usually avoided or converted early.

Scalability: JSON is **scalable in a horizontal sense** – because it is text, it compresses well (one can gzip JSON data to significantly reduce transmission size, and JSON compressibility is high due to repeated keys and structure). Also, JSON being schema-free means adding new fields doesn't break old code (unless they are huge), which helps scalability of evolving AI systems. On the other hand, extremely deep or large JSON structures can cause issues. For instance, a deeply nested JSON (recursive objects within objects) could lead to stack overflow in some parsers or extreme memory usage; some parsers set limits on depth to prevent malicious inputs from consuming resources ²⁸. Also, very large numeric arrays in JSON are inefficient – if an AI model outputs a large vector or matrix, sending it as JSON array of numbers will be bulky. In those cases, binary formats or at least base64 encoding binary data is considered. But base64 inflates size ~33%, so even that is a compromise. Some AI deployments use **mixed formats**: e.g., for an image classification service, the image is sent as a JPEG (binary) and the response is JSON. Or a training pipeline might keep raw feature data in binary files and only use JSON for higher-level metadata linking those files.

Finally, **parallelism and JSON**: Because JSON must be parsed sequentially, multi-threaded parsing of a single JSON document is not straightforward (except in advanced libraries that split the file and use SIMD as mentioned). This contrasts with, say, CSV or plain text where you can sometimes split by newline and parse chunks independently (though you still need to assemble the final structure). For extremely large JSON, one strategy is to split the file into chunks at known boundaries (if the JSON is an array, you can split at item boundaries) and parse in parallel. Another strategy is to avoid single gigantic JSON files; instead, store many small JSON files (e.g., one per data record). This can make retrieval and parsing more manageable in distributed training scenarios (some AI training pipelines use one JSON per data point, which can be loaded independently, at the cost of file overhead).

In summary, JSON carries some performance overhead due to being text and richly structured, but these are manageable for most AI uses with modern computing resources. Techniques like streaming, compression, and careful batching mitigate many issues. For truly large-scale or real-time low-latency systems, engineers might opt for alternative encodings or limit JSON's use to control overhead. Nonetheless, the **flexibility and universality of JSON often justify its use** despite performance costs. As one commentary notes, JSON's human-readability is “overhead” that might be unnecessary for machine-only communication ²⁹, but in AI development and deployment, there's value in that transparency during development and debugging. The key is to use JSON where it provides benefit and not for scenarios where its overhead would dominate (like massive tensor data or high-frequency microsecond-level IPC).

Limitations and Security Aspects of JSON in AI Systems

While JSON is a convenient data format, it has inherent limitations and some security considerations that practitioners should keep in mind when using it in AI systems.

Expressiveness Limitations: JSON's type system is minimal, which can be a limitation for complex data. There is **no native date/time type**, no binary type (raw bytes must be encoded as text, usually Base64), and as discussed, numeric values are all treated as one generic "number" type (usually floating-point). This means that important distinctions can be lost or require convention to interpret. For example, a timestamp might be stored as a string (ISO 8601 format) or as a number (Unix epoch), but JSON itself doesn't know it's a date – the interpretation happens in the application layer. In AI pipelines, this can lead to subtle bugs if not documented: e.g., a model configuration might have `"learning_rate": "1e-4"` as a string versus `1e-4` as a number, which in JSON are different types (the former could be ignored by a parser expecting a numeric). Another limitation is precision: JSON numbers are often **limited to double precision** in practical use¹⁴. If an AI application needs arbitrary precision integers (for cryptographic protocols or exact counters) or higher precision floats, JSON cannot guarantee to carry those accurately. An integer with 20+ digits might get rounded or represented in scientific notation and lose precision when parsed by a typical JSON library. This is rarely a concern in core ML tasks, but in certain data processing it might be.

No Schema/Comments: JSON on its own does not enforce a schema. For AI data interchange, this means senders and receivers must have an **out-of-band agreement** on the JSON structure (or use JSON Schema definitions to validate). Unlike some formats (XML with XSD, or Protocol Buffers with .proto definitions), JSON won't prevent missing fields or mis-typed values by itself. In practice, many AI systems implement runtime checks or use libraries to validate JSON inputs (for instance, ensuring that a prediction request JSON has all required fields and correct types). The lack of comments in JSON is another oft-cited limitation – configuration files in JSON cannot include human-readable commentary (developers sometimes work around this by adding unused fields like `"_comment": "text"` which is generally considered a hack). This has led to the adoption of alternative formats like YAML or TOML for configuration where human-maintained files need comments. In the context of AI, if researchers share a JSON config for a model, they might have to maintain separate documentation for what each field means, rather than annotating inline. The strict syntax (no trailing commas, no comments) makes JSON less forgiving for human editing, which is a limitation in iterative development.

Interoperability Pitfalls: Although JSON is standardized, there have been interoperability pitfalls. One historical example is that JSON text used to be allowed in various encodings (UTF-8, UTF-16, etc.), but the newer standard now requires UTF-8 for network transfer¹⁰. If an AI system erroneously outputs JSON in another encoding, it might break downstream. Another subtle issue is **Big numbers**: some JSON implementations might serialize a very large integer as a numeric literal, but the receiver's JSON parser could truncate it or treat it as string if out of range. For instance, a JSON like `{"total": 9007199254740993}` (which is $2^{53}+1$) might parse differently in languages depending on their number handling (JavaScript would lose precision since it uses 64-bit float for numbers). These are not security issues per se, but can lead to data inaccuracies – an AI audit system might miscalculate totals if such big numbers are involved.

Security Considerations: JSON is generally considered safer than earlier data formats like XML in terms of injection attacks, but it is not without risks. The IETF RFC 8259 notes that since JSON's syntax is borrowed from JavaScript, it's possible (in a JavaScript environment) to use the `eval()` function on a JSON string, but

doing so is **dangerous and strongly discouraged** ³⁰. If an attacker manages to supply malicious JSON and a program naively does `eval(malicious_json)`, they could execute arbitrary code. The safe practice is to use a proper JSON parser, which will only create data structures and not execute code. This is a well-known web security point: always use `JSON.parse()` in JavaScript, not `eval`. Similarly, in Python or other languages, one should not use general deserialization on JSON (like Python's `eval` or `ast.literal_eval` on untrusted input) – instead, use a JSON library that does not have execution capabilities. Fortunately, JSON parsers in most languages simply create dictionaries/objects and are safe against code execution by design.

Another possible security issue is **JSON injection**. If an AI system constructs JSON responses by concatenating strings (instead of using a library), and if it embeds user-provided data, it could create invalid or malicious structures. For example, if user input is directly inserted into a JSON response without sanitization, they might inject additional fields or break the JSON format. This risk is mitigated by always using proper serialization functions that will escape characters as needed. Web APIs also set the `Content-Type: application/json` header so that browsers treat JSON as data. In the past, there was a concept of “JSON Hijacking” in early AJAX applications – if a sensitive JSON was accessible via a GET request, a malicious page could potentially load it via a `<script>` tag (since JSON is valid JavaScript syntax for objects/arrays) and steal the data. To counter this, some APIs used to prefix JSON responses with a non-JSON lead (while sending as text) or require POST. Nowadays, setting the correct MIME type and CORS headers prevents JSON from being executed as script in the browser, so this is less of a concern, but it's part of the historical security landscape of JSON.

Denial of Service (DoS) Concerns: Like any format, JSON parsing can be abused to exhaust resources. An adversary might send a **huge JSON payload** (many millions of nested elements) to an AI service expecting JSON input, trying to crash it or make it hang. To address this, JSON parsers or the application should enforce limits: maximum size of input, maximum nesting depth ²⁸, and perhaps maximum number of array elements or object members. Many JSON libraries allow configuring a size limit or will throw an error if depth is excessive. For example, a recursive 10000-level nested array could overflow the stack in some languages. Therefore, AI systems ingesting JSON from external sources should have checks in place (e.g., rejecting inputs that are obviously too large or deeply nested). In training pipelines, if JSON files are loaded from untrusted sources (say a user uploads a JSON of annotations), similar caution is warranted.

Data Integrity and Validation: JSON's lack of a schema means that *invalid or unexpected data could slip through* unless explicitly checked. In an AI system, imagine a JSON configuration is missing a crucial field (say, `"learning_rate"` is absent). The training code might either assume a default (potentially leading to unnoticed differences in experiments) or crash later with a confusing error. To enhance security and robustness, teams use JSON Schema or manual validation to ensure that incoming JSON conforms to what is expected. JSON Schema is a standard that lets you define required keys, their types, ranges, etc., and can be used to validate JSON files (for instance, one could define a schema for a model's config file and validate before using it, to catch any user error or tampering).

Comparison to PDF and TXT security: JSON does not carry active content like macros, which is a plus. PDFs can contain embedded scripts or links, raising issues like PDF-based malware or the need for sandboxing PDF readers. Plain text files are generally passive, though if an AI system interprets text in commands, it could be tricked (but that's more about the system using the text). JSON's simplicity means fewer potential vectors – it doesn't support entity expansion like XML (avoiding the billion laughs attack), and parsers typically handle recursion gracefully or error out. However, one should note that if an AI system stores JSON

and later retrieves it, there should be proper encoding/escaping if that JSON is then embedded in HTML or a shell context to avoid injection in those contexts (standard web sanitization practices).

In summary, **JSON is relatively secure by design** (data-only, no executable aspects) but must be used with good practices: always parse with vetted libraries (never with `eval`), validate structure to avoid unexpected conditions, and guard against resource exhaustion with limits. The limitations of JSON (types, precision, lack of comments) are generally manageable in AI workflows, but awareness of them prevents certain classes of bugs. For instance, if an AI model output includes a `NaN` (from a calculation) – since JSON cannot represent that, the system might convert it to `null` or string, which the downstream must handle. Being mindful of such edge cases (and documenting them in API docs or data schemas) is part of building robust AI data pipelines. Despite these caveats, JSON remains a cornerstone format because its benefits in interoperability and ease-of-use typically outweigh the downsides, especially when mitigated by careful engineering.

Conclusion

JSON has proven to be a versatile and powerful format for data representation in artificial intelligence contexts. Its **full syntax** – comprising a mere handful of structural symbols and base types – is simple yet expressive enough to model complex data structures used in AI, from nested annotations to model configurations. We have seen that JSON's human-readability and self-describing nature facilitate transparency in data storage and exchange, which is valuable during model development, debugging, and integration. In **AI model training pipelines**, JSON is frequently employed for labeling data and specifying experiments, providing a common thread from raw data to configuration files. For **model deployment and inference**, JSON serves as the lingua franca in REST APIs, enabling seamless communication between models and applications. Our technical comparison underscored that unlike PDFs (which are optimized for human viewing) or plain text (unstructured streams of characters), JSON hits a sweet spot of being both machine-friendly and reasonably human-comprehensible – a trait that has undoubtedly contributed to its ubiquity.

However, using JSON in AI systems is not without challenges. In terms of **performance and scalability**, we noted that JSON's text-based format incurs parsing costs and overhead, which can impact high-scale systems. Strategies such as streaming (NDJSON) and careful batching can alleviate these issues, but practitioners should remain cognizant of JSON's limitations with very large or numerous messages. Moreover, we discussed how JSON's inherent **limitations** (limited data types, no built-in schema, no support for special numeric values, etc.) require developers to enforce discipline via schemas or code checks to maintain data integrity in AI workflows. On the **security** front, JSON is generally benign, but we highlighted the importance of using proper parsing (to avoid code injection via `eval`) and validating inputs to protect against maliciously crafted payloads or resource exhaustion attacks³⁰. By adhering to standard best practices – much of which are documented in the JSON specification and related literature – AI engineers can safely harness JSON's capabilities.

In conclusion, JSON's role in artificial intelligence is firmly established: it is the **glue that connects components** – from data preprocessing, to model configuration, to inference result delivery. Its design aligns well with the needs of AI systems that must handle structured yet dynamic data across different programming environments. While alternatives like binary serialization, PDFs, or raw text each have their niches, JSON offers a balanced mix of structure and flexibility that has become integral to modern AI pipelines. As AI systems continue to evolve, JSON's broad adoption suggests it will remain a foundational tool, though it may be augmented by schemas and complementary binary formats where performance is

paramount. The key for practitioners is to leverage JSON where appropriate, mindful of its specs and leveraging its strengths (interoperability, clarity) while mitigating its weaknesses (performance overhead, lack of strong typing) with robust engineering. In the ever-expanding landscape of AI technology, JSON stands out as a classic solution that continues to enable systems to “speak” a common language of data ¹ – a crucial factor in collaborative, scalable, and transparent AI development.

References:

- T. Bray, *The JavaScript Object Notation (JSON) Data Interchange Format*, IETF RFC 8259, 2017 ¹ ² ³ ¹⁰ ³⁰ .
 - A. Monus, "SOAP vs REST vs JSON – a 2023 comparison," *Raygun Blog*, Oct. 17, 2022 ¹⁷ .
 - Roboflow, "What is the COCO JSON Annotation Format?" (Web article) ¹⁶ .
 - J. Brownlee, *How to Save and Load Your Keras Deep Learning Model*, MachineLearningMastery, 2023 ¹⁸ .
 - TensorFlow Documentation, *TensorFlow Serving RESTful API Guide*, accessed 2025 ¹⁹ .
 - I. Montani, "From PDFs to AI-ready structured data: a deep dive," *Explosion AI Blog*, Dec 2, 2024 ²¹ .
 - O. Pereverzieva, "Structured vs Unstructured Data Explained," *AltexSoft Blog*, Dec 16, 2024 ³¹ ³² .
 - S. McCarthy, "JSON is dangerous (and slow)," *Personal Blog*, Jan 23, 2024 ²⁴ ²⁰ .
 - MongoDB, "JSON and BSON" (MongoDB Guides), MongoDB Inc. ¹⁵ .
 - *JSON streaming* (Wikipedia article: Newline-delimited JSON), accessed 2025 ²⁶ .
- (Additional sources cited inline)

¹ ² ³ ⁴ ⁶ ⁷ ⁸ ¹⁰ ¹¹ ¹² ²⁸ ³⁰ RFC 8259 - The JavaScript Object Notation (JSON) Data Interchange Format

<https://datatracker.ietf.org/doc/html/rfc8259>

⁵ ¹³ ¹⁴ ²⁰ ²² ²⁴ ²⁵ ²⁷ ²⁹ JSON is dangerous (and slow) - Smári McCarthy

<https://smarimccarthy.is/posts/2024-01-23-json-bad/>

⁹ Trailing commas - JavaScript | MDN

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Trailing_commas

¹⁵ JSON And BSON | MongoDB | MongoDB

<https://www.mongodb.com/resources/basics/json-and-bson>

¹⁶ What is the COCO JSON Annotation Format?

<https://roboflow.com/formats/coco-json>

¹⁷ SOAP vs REST vs JSON - a 2023 comparison · Raygun Blog

<https://raygun.com/blog/soap-vs-rest-vs-json/>

¹⁸ How to Save and Load Your Keras Deep Learning Model - MachineLearningMastery.com

<https://machinelearningmastery.com/save-load-keras-deep-learning-models/>

¹⁹ RESTful API | TFX | TensorFlow

https://www.tensorflow.org/tfx/serving/api_rest

²¹ ²³ From PDFs to AI-ready structured data: a deep dive · Explosion

<https://explosion.ai/blog/pdfs-nlp-structured-data>

26 JSON streaming - Wikipedia

https://en.wikipedia.org/wiki/JSON_streaming

31 32 Structured vs Unstructured Data Explained with Examples

<https://www.altexsoft.com/blog/structured-unstructured-data/>